

Section 4.3 — Managing Cloud Run Resources

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 4: Ensuring successful operation of a cloud solution (~20 % of the exam)**. You must know how to deploy new versions, manage traffic splitting, and configure scaling parameters.

1. Cloud Run Service Model

Key Concepts

```
Cloud Run Service
├─ Revision 1 (v1) — old version — 0% traffic
├─ Revision 2 (v2) — previous version — 10% traffic (canary)
└─ Revision 3 (v3) — latest version — 90% traffic (production)
```

- **Service** — The top-level resource that manages revisions and routes traffic
- **Revision** — An immutable snapshot of a service configuration (code + settings)
- **Instance** — A running container handling requests
- Each deployment creates a **new revision**
- Traffic can be split across revisions

Service Lifecycle

```
# List Cloud Run services
gcloud run services list --region=us-central1

# Describe a service
gcloud run services describe my-service --region=us-central1

# Get the URL of a service
gcloud run services describe my-service \
  --region=us-central1 \
  --format="value(status.url)"

# Delete a service
gcloud run services delete my-service --region=us-central1
```

2. Deploying New Versions of an Application

Standard Deployment (New Revision)

Every `gcloud run deploy` creates a new revision. By default, the new revision receives **100% of traffic**.

```
# Deploy a new version (all traffic goes to new revision)
gcloud run deploy my-service \
  --image=us-central1-docker.pkg.dev/PROJECT/REPO/my-app:v2 \
  --region=us-central1

# Deploy without routing traffic (test the new revision first)
gcloud run deploy my-service \
  --image=us-central1-docker.pkg.dev/PROJECT/REPO/my-app:v2 \
  --region=us-central1 \
  --no-traffic

# Deploy with a revision tag (creates a named URL for testing)
gcloud run deploy my-service \
  --image=us-central1-docker.pkg.dev/PROJECT/REPO/my-app:v2 \
  --region=us-central1 \
  --no-traffic \
  --tag=canary
# Access at: https://canary---my-service-xxx-uc.a.run.app
```

Updating Service Configuration (Without Changing Image)

```
# Update environment variables
gcloud run services update my-service \
  --region=us-central1 \
  --set-env-vars="DB_HOST=new-host,LOG_LEVEL=info"

# Add an environment variable without removing existing ones
gcloud run services update my-service \
  --region=us-central1 \
  --update-env-vars="NEW_VAR=value"

# Remove an environment variable
gcloud run services update my-service \
  --region=us-central1 \
  --remove-env-vars="OLD_VAR"

# Update memory and CPU
gcloud run services update my-service \
  --region=us-central1 \
  --memory=1Gi \
  --cpu=2

# Update the service account
gcloud run services update my-service \
  --region=us-central1 \
  --service-account=new-sa@PROJECT.iam.gserviceaccount.com

# Update concurrency
gcloud run services update my-service \
  --region=us-central1 \
  --concurrency=100

# Update timeout
gcloud run services update my-service \
  --region=us-central1 \
  --timeout=600
```

Viewing Revisions

```
# List all revisions for a service
gcloud run revisions list \
  --service=my-service \
  --region=us-central1

# Describe a specific revision
gcloud run revisions describe REVISION_NAME \
  --region=us-central1

# Delete an old revision (only if it has 0% traffic)
gcloud run revisions delete REVISION_NAME \
  --region=us-central1
```

3. Traffic Splitting Parameters

Traffic splitting enables **canary deployments**, **gradual rollouts**, and **instant rollbacks**.

Splitting Traffic Between Revisions

```
# Route all traffic to a specific revision
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00003-abc=100

# Split traffic 90/10 (canary deployment)
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00002-xyz=90,my-service-00003-abc=10

# Gradual rollout
# Step 1: 95/5
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00002-xyz=95,my-service-00003-abc=5

# Step 2: 80/20
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00002-xyz=80,my-service-00003-abc=20

# Step 3: 50/50
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00002-xyz=50,my-service-00003-abc=50

# Step 4: 0/100 (complete migration)
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00003-abc=100

# Route all traffic to the latest revision
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-latest
```

Using Tags for Testing

```
# Tag a revision (creates a named URL for testing)
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --set-tags=canary=my-service-00003-abc

# The tagged revision is accessible at:
# https://canary---my-service-xxx-uc.a.run.app
# This URL doesn't affect the main traffic split

# Send a percentage of traffic to a tag
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-tags=canary=10

# Remove a tag
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --clear-tags
```

Rollback

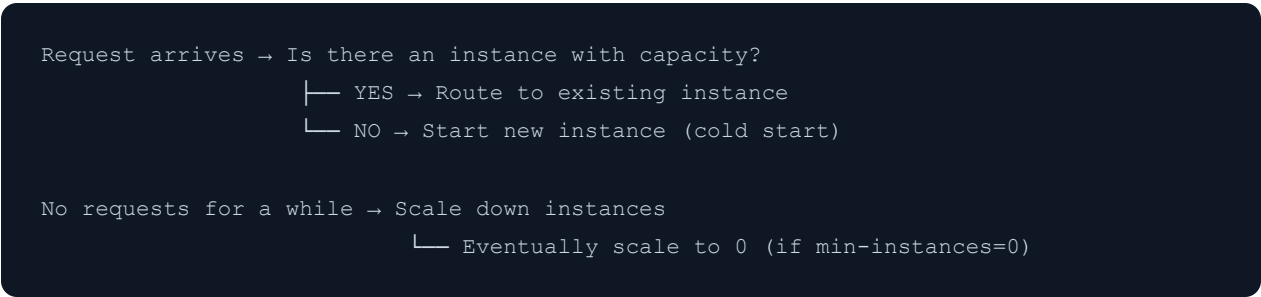
```
# Instant rollback: Route 100% traffic to the previous revision
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00002-xyz=100
```

View Current Traffic Distribution

```
# See traffic split
gcloud run services describe my-service \
  --region=us-central1 \
  --format="yaml(status.traffic)"
```

4. Scaling Parameters for Autoscaling Instances

How Cloud Run Autoscaling Works



Scaling Parameters

Parameter	Description	Default	Range
<code>--min-instances</code>	Minimum instances always running	0	0-1000
<code>--max-instances</code>	Maximum instances to scale to	100	1-1000
<code>--concurrency</code>	Max concurrent requests per instance	80	1-1000
<code>--cpu-throttling</code>	Throttle CPU outside request processing	Yes	Yes/No

Configuring Scaling

```
# Set min and max instances
gcloud run deploy my-service \
  --image=IMAGE \
  --region=us-central1 \
  --min-instances=2 \
  --max-instances=50

# Update scaling on existing service
gcloud run services update my-service \
  --region=us-central1 \
  --min-instances=1 \
  --max-instances=100

# Set concurrency (requests per instance)
gcloud run services update my-service \
  --region=us-central1 \
  --concurrency=50

# Disable CPU throttling (CPU always allocated)
gcloud run services update my-service \
  --region=us-central1 \
  --no-cpu-throttling

# Scale to zero (no minimum instances)
gcloud run services update my-service \
  --region=us-central1 \
  --min-instances=0
```

Understanding min-instances

Setting	Behavior	Cost	Use Case
min-instances=0	Scale to zero when idle	Lowest (no cost when idle)	Dev/test, infrequent traffic
min-instances=1	1 instance always warm	Low (minimal idle cost)	Avoid cold starts for light traffic
min-instances=N	N instances always ready	Higher (N instances running)	Production with latency SLAs

Understanding Concurrency

- **Concurrency** = max simultaneous requests one instance handles

- Higher concurrency → fewer instances needed → lower cost
- Lower concurrency → more isolation → higher cost
- Optimal setting depends on your application's resource usage per request

```
Example: 100 requests/second with concurrency=50  
→ Need ~2 instances (100/50 = 2)
```

```
Example: 100 requests/second with concurrency=1  
→ Need ~100 instances (100/1 = 100)
```

CPU Allocation Options

Option	CPU Behavior	Pricing	Use Case
CPU throttled (default)	CPU allocated only during request processing	Per request	Standard web services
CPU always allocated	CPU available even between requests	Per instance time	Background processing, WebSockets, long-lived connections

```
# CPU always allocated  
gcloud run services update my-service \  
  --region=us-central1 \  
  --no-cpu-throttling  
  
# CPU throttled (default)  
gcloud run services update my-service \  
  --region=us-central1 \  
  --cpu-throttling
```

5. Cloud Run IAM and Authentication

Access Control

```
# Allow unauthenticated access (public)
gcloud run services add-iam-policy-binding my-service \
  --region=us-central1 \
  --member="allUsers" \
  --role="roles/run.invoker"

# Restrict to authenticated users only
gcloud run services remove-iam-policy-binding my-service \
  --region=us-central1 \
  --member="allUsers" \
  --role="roles/run.invoker"

# Grant a specific user/service account invoke access
gcloud run services add-iam-policy-binding my-service \
  --region=us-central1 \
  --member="serviceAccount:my-sa@PROJECT.iam.gserviceaccount.com" \
  --role="roles/run.invoker"

# Grant a group invoke access
gcloud run services add-iam-policy-binding my-service \
  --region=us-central1 \
  --member="group:developers@example.com" \
  --role="roles/run.invoker"
```

6. Cloud Run Logging and Monitoring

```
# View service logs
gcloud run services logs read my-service --region=us-central1

# Tail logs in real-time
gcloud run services logs tail my-service --region=us-central1

# View logs for a specific revision
gcloud run revisions logs read REVISION_NAME --region=us-central1
```

Key Metrics Available in Cloud Monitoring

- **Request count** — Total requests by response code
- **Request latency** — P50, P95, P99 latencies

- **Instance count** — Number of running instances
 - **CPU utilization** — Average CPU usage
 - **Memory utilization** — Average memory usage
 - **Billable instance time** — Total billed time
 - **Startup latency** — Cold start time
-

7. VPC Connector and Private Networking

- Cloud Run is fully managed and runs outside your VPC by default
- To reach private resources (Cloud SQL private IP, Memorystore, internal VMs): use Serverless VPC Access connector

```
# Create connector
gcloud compute networks vpc-access connectors create my-connector \
  --region=us-central1 \
  --subnet=my-subnet \
  --subnet-project=MY_PROJECT \
  --min-instances=2 \
  --max-instances=10
# Deploy Cloud Run with connector
gcloud run deploy MY_SERVICE \
  --image=IMAGE \
  --vpc-connector=my-connector \
  --vpc-egress=private-ranges-only \
  --region=us-central1
```

- `--vpc-egress=all-traffic` : routes ALL outbound traffic through VPC (for static outbound IP via Cloud NAT)
 - `--vpc-egress=private-ranges-only` : only RFC 1918 traffic goes through VPC; public traffic goes direct
-

8. Secret Manager Integration

- Inject secrets as environment variables or mounted volumes at deploy time:

```
gcloud run deploy MY_SERVICE \
  --image=IMAGE \
  --set-secrets=DB_PASSWORD=my-secret:latest \
  --set-secrets=API_KEY=another-secret:2 \
  --region=us-central1
```

- The Cloud Run service account must have `roles/secretmanager.secretAccessor` on the secret
- Mounted volume:

```
gcloud run deploy MY_SERVICE --set-secrets=/secrets/db=my-secret:latest
```

- Exam tip: Prefer Secret Manager over environment variables for credentials — secrets update without redeployment when using volume mounts

Exam Practice Questions

1. **You deployed a new version of your Cloud Run service but want to test it before sending production traffic. How?**
 - Answer: Deploy with `--no-traffic` and `--tag=canary`. Test using the tag URL (`https://canary---service-xxx.a.run.app`). When satisfied, route traffic with `gcloud run services update-traffic --to-tags=canary=100`.
2. **Your Cloud Run service experiences cold starts that impact user experience. How can you reduce this?**
 - Answer: Set `--min-instances=1` (or higher) to keep warm instances always running. This eliminates cold starts for the first N concurrent requests.
3. **You need to roll back a Cloud Run service to the previous version immediately. What's the fastest way?**
 - Answer: `gcloud run services update-traffic my-service --to-revisions=PREVIOUS_REVISION=100`. This instantly routes all traffic back to the previous revision.
4. **Your service handles long-lived WebSocket connections. Should you use CPU throttling?**
 - Answer: **No**. Use `--no-cpu-throttling` so CPU is always allocated, even between request processing. This is required for WebSockets and background processing.
5. **What happens to instances when there are no requests and min-instances is set to 0?**

- Answer: Cloud Run **scales to zero** — all instances are terminated. No charges accrue. When a new request arrives, a new instance is started (cold start).

6. You want to split traffic 80/20 between two revisions for an A/B test. How do you configure this?

- Answer: `gcloud run services update-traffic my-service --to-revisions=revision-a=80,revision-b=20 --region=us-central1`.

Glossary

A/B Test — A deployment strategy that routes a defined percentage of traffic to two different service revisions simultaneously to compare behavior or performance.

Artifact Registry — GCP's managed repository service for storing container images; Cloud Run pulls its deployment images from Artifact Registry repositories.

Canary Deployment — A release strategy in which a small percentage of traffic is routed to a new revision while the majority continues to reach the stable version, allowing controlled testing before full rollout.

Cloud Monitoring — GCP's managed monitoring service that collects and visualizes metrics for Cloud Run services, including request count, latency, instance count, and CPU/memory utilization.

Cloud NAT — A GCP managed service that provides outbound internet connectivity for Cloud Run instances routing all traffic through a VPC, enabling a static outbound IP address.

Cloud Run — A fully managed GCP serverless compute platform that automatically scales stateless containers in response to HTTP requests, events, or other triggers.

Cloud SQL — GCP's fully managed relational database service; Cloud Run services access Cloud SQL private IP endpoints via a Serverless VPC Access connector.

Cold Start — The latency experienced when a Cloud Run instance must be initialized from scratch to handle an incoming request, occurring when no warm instances are available.

Concurrency — The maximum number of simultaneous requests that a single Cloud Run instance is configured to handle; higher concurrency reduces the number of instances required.

CPU Throttling — The default Cloud Run behavior that allocates CPU only during request processing, reducing cost; can be disabled with `--no-cpu-throttling` for background workloads.

CPU Utilization — A Cloud Monitoring metric tracking the average percentage of CPU used by Cloud Run instances; available for monitoring and alerting.

Environment Variable — A key-value configuration setting injected into a Cloud Run container at deploy time; used to pass configuration values like `DB_HOST` and `LOG_LEVEL`.

GCP (Google Cloud Platform) — Google's suite of cloud computing services, including Cloud Run as its fully managed serverless container platform.

gcloud — The primary command-line tool for interacting with GCP services; used to deploy, update, and manage Cloud Run services and revisions.

Gradual Rollout — A deployment approach that incrementally increases the traffic percentage directed to a new revision over multiple steps, minimizing risk.

IAM (Identity and Access Management) — GCP's access control system; used to grant or restrict invocation rights on Cloud Run services via roles like `roles/run.invoker`.

Instance — A running container in Cloud Run that actively handles one or more requests; Cloud Run automatically creates and destroys instances based on traffic.

Memorystore — GCP's fully managed in-memory data store service (Redis/Memcached); accessible from Cloud Run via a Serverless VPC Access connector.

Memory Utilization — A Cloud Monitoring metric tracking average memory usage of Cloud Run instances.

min-instances — A Cloud Run scaling parameter that sets the minimum number of instances to keep running at all times, preventing cold starts at the cost of idle instance charges.

max-instances — A Cloud Run scaling parameter that caps the maximum number of instances Cloud Run may scale to, preventing runaway costs under unexpected traffic spikes.

P50 / P95 / P99 Latency — Percentile-based latency metrics: P50 is the median, P95 means 95% of requests are faster, P99 means 99% of requests are faster; used to measure tail latency.

RFC 1918 — An Internet standard defining private IP address ranges; Cloud Run's `--vpc-egress=private-ranges-only` routes only RFC 1918 traffic through the VPC connector.

Revision — An immutable snapshot of a Cloud Run service configuration (container image, environment variables, scaling settings, etc.); each deployment creates a new revision.

roles/run.invoker — The GCP IAM role that grants the ability to invoke (send requests to) a Cloud Run service; assigned to `allUsers` for public services or specific identities for private ones.

roles/secretmanager.secretAccessor — The GCP IAM role that allows a service account to access and read secret values from Secret Manager.

Rollback — The process of instantly redirecting 100% of traffic back to a previous stable revision of a Cloud Run service to recover from a bad deployment.

Secret Manager — A GCP service for securely storing and managing sensitive credentials; integrated with Cloud Run to inject secrets as environment variables or mounted volumes.

Serverless VPC Access — A GCP connector resource that enables Cloud Run (and other serverless products) to communicate with resources inside a VPC network using private IP addresses.

Service — The top-level Cloud Run resource that manages one or more revisions and controls how traffic is routed among them via a stable URL.

Service Account — A GCP identity assigned to a Cloud Run service to authenticate outbound API calls to other GCP services without storing user credentials.

SLA (Service Level Agreement) — A contractual availability commitment; Cloud Run provides defined SLAs that influence decisions around `min-instances` and traffic splitting strategies.

Tag — A named label applied to a specific Cloud Run revision that creates a dedicated URL for testing purposes, independent of the main traffic split.

TCP (Transmission Control Protocol) — A connection-oriented transport protocol; relevant to Cloud Run for WebSocket connections and long-lived TCP connections requiring `--no-cpu-throttling`.

Traffic Splitting — A Cloud Run feature that distributes incoming requests across multiple revisions by percentage, enabling canary deployments, gradual rollouts, and A/B testing.

VPC (Virtual Private Cloud) — A logically isolated network in GCP; Cloud Run services connect to a VPC using a Serverless VPC Access connector to reach private resources.

VPC Connector — See Serverless VPC Access; the resource provisioned via `gcloud compute networks vpc-access connectors create` that bridges Cloud Run to a VPC subnet.

WebSocket — A protocol enabling full-duplex communication over a persistent TCP connection; requires Cloud Run to use `--no-cpu-throttling` so CPU remains allocated between messages.

Warm Instance — A Cloud Run container instance that is already running and ready to accept requests immediately, avoiding cold start latency.